

SDET Technical Assessment – API Test Automation Exercise

Objective

The purpose of this exercise is to evaluate your ability to:

- Analyze an API surface and identify test scenarios.
- Design and implement automated API tests.
- Structure a maintainable test automation project.
- Demonstrate coding and testing best practices.
- Document and communicate your solution clearly.

Time Limit

This exercise is strictly time-boxed to **2 hours total**.

We are intentionally evaluating how you prioritize work and make trade-offs under time constraints. We do not expect exhaustive coverage of every possible test scenario.

API Under Test

Use the publicly available JSONPlaceholder API:

<https://jsonplaceholder.typicode.com>

Your objective is to identify and automate tests for **all available API routes/resources** exposed by the service.

Examples include (but are not limited to):

- /posts
- /comments
- /albums
- /photos
- /todos
- /users

You are encouraged to explore the API and determine appropriate test coverage. Note that the publicly accessible JSONPlaceholder API does not persist data (however, POST/PUT/PATCH operations can still be executed successfully).

Technology Requirements

Preferred languages:

- Python
- C#

Other programming languages are acceptable if they are commonly used for automated testing. You may use any testing framework and supporting libraries you feel are appropriate.

Source Control Requirements

Your solution must be submitted through a public GitHub repository.

Timing Verification

To verify adherence to the 2-hour time limit:

1. Create a new GitHub repository.
2. Make an **initial commit** before beginning development.
3. Begin work only after the initial commit has been pushed.
4. Complete the exercise within 2 hours.
5. Make a **final commit** when you stop working.
6. Push all commits to GitHub.

The commit history should clearly demonstrate the start and end of your work period.

You may make additional commits during development if desired.

Deliverables

Your repository should contain:

Automated Tests

Implement API tests that you believe provide the highest value within the allotted time.

Examples of areas to consider:

- Positive test cases.
- Response validation.
- Status code verification.
- Edge cases.
- Parameterized tests.
- Negative scenarios (where applicable).

Documentation

Include a README.md containing:

Overview

- Brief description of the solution.
- Assumptions made.
- Scope of testing completed.

Execution Instructions

Document exactly how to:

- Install dependencies.
- Build the project (if applicable).
- Execute the test suite.
- View test results.

Coverage Summary

Provide a short summary of:

- Routes/resources tested.
- Types of validations implemented.
- Areas intentionally omitted due to time constraints.

Important Notes

- The goal is not to achieve 100% coverage.
- Focus on demonstrating sound engineering and testing practices.
- We value thoughtful design and maintainable code over sheer quantity.
- AI-assisted development tools may be used; however, be prepared to discuss and explain all code submitted.
- Please ensure the repository is accessible to the interview team.